

Department of Computer and Software Engineering**SE: Machine Learning****Course Instructor:****Date:****Day:****Semester:****Lab 10: Model Optimization and Feature Engineering**

Lab Title	CLO	BT	PLO	Weightage
Model Optimization & Feature Engineering CV, Optuna, Encoding & Leakage				

Name	Roll Number	Report /10	Viva /5	Total /15
Zunaira Abdul Aziz	BSSE23058			

Checked on: _____

Signature: _____

Objectives

- Apply Grid Search, Random Search, and Bayesian Optimization
- Understand and use Cross-Validation properly
- Implement Target Encoding and avoid leakage
- Identify and fix Data Leakage
- Engineer Cyclical Features

Problem Statement

You are given a dataset for a binary classification problem. The dataset contains numerical, categorical, and temporal features.

Your goal is to improve model performance using optimization techniques and feature engineering while ensuring that no data leakage occurs.

Dataset File: lab10_data.csv

The dataset contains:

- Numerical features: age, income
- Categorical feature: city
- Temporal feature: hour (0–23)
- Target variable: binary classification
- A hidden leakage feature for analysis

Dataset: Customer Transaction Response Prediction

This dataset simulates a real-world customer behavior prediction problem commonly encountered in domains like digital marketing, fraud detection, recommendation systems, and customer analytics.

Each row represents a user interaction or transaction event, and the goal is to predict a binary outcome (e.g., whether a user converts, responds, or triggers a flag).

Feature Description

Numerical Features

Numerical features include **age**, representing the age of the customer (18–60), which often correlates with purchasing power, risk behavior, or engagement; and **income**, a simulated

annual income with realistic variance where higher-income users tend to behave differently in spending, fraud patterns, or response likelihood.

Categorical Feature

The categorical feature **city** represents customer location segment (A, B, C) and in practice could correspond to geographic regions, customer tiers, or market segments. This feature is useful for demonstrating target encoding, categorical bias, and leakage pitfalls.

Temporal Feature (Cyclical)

The temporal feature **hour (0–23)** represents the hour of the event (e.g., transaction time); in real-world systems user behavior is strongly time-dependent, fraud often spikes at unusual hours, and engagement varies across the day. This feature is intentionally included to teach why raw hour is a poor representation and why cyclical encoding (sin/cos) is necessary.

Target Variable

The **target (0 or 1)** represents a binary outcome such as fraud vs normal transaction, click vs no click, or conversion vs no conversion. The target is not purely random—it depends on income patterns and time-based behavior (hour), ensuring models can learn meaningful patterns and that optimization techniques show measurable improvements.

Part A: Hyperparameter Optimization

Task 1: Grid Search

Use GridSearchCV to tune a Decision Tree model.

Tune: - max depth - min samples split

Report the best parameters.

```
def run_grid_search(X, y):
    param_grid = {
        'max_depth': [3, 5, 10, None],
        'min_samples_split': [2, 5, 10]
    }
    grid_search = GridSearchCV(DecisionTreeClassifier(), param_grid,
cv=5)
    grid_search.fit(X, y)
    print(f"Grid Search Best Params: {grid_search.best_params_}")
    return grid_search.best_estimator_
```

Task 2: Random Search

Use RandomizedSearchCV.

Compare results and runtime with Grid Search.

```
def run_random_search(X, y):
    param_dist = {
        'max_depth': [int(x) for x in np.linspace(3, 20, 10)],
        'min_samples_split': [2, 5, 10, 20]
    }
    start_time = time.time()
    random_search = RandomizedSearchCV(DecisionTreeClassifier(),
param_dist, n_iter=10, cv=5)
    random_search.fit(X, y)
    duration = time.time() - start_time
    print(f"Random Search Best Params: {random_search.best_params_}
(Runtime: {duration:.2f}s)")
    return random_search.best_estimator_
```

Task 3: Bayesian Optimization (Optuna)

Optimize hyperparameters using Optuna.

Run at least 20 trials and report best parameters.

```
def run_optuna(X, y):
    def objective(trial):
        max_depth = trial.suggest_int("max_depth", 2, 32, log=True)
        min_samples_split = trial.suggest_int("min_samples_split", 2, 20)

        model = DecisionTreeClassifier(max_depth=max_depth,
min_samples_split=min_samples_split)
        score = cross_val_score(model, X, y, n_jobs=-1, cv=3).mean()
        return score

    study = optuna.create_study(direction="maximize")
    study.optimize(objective, n_trials=20)
    print(f"Optuna Best Params: {study.best_params}")
    return study.best_params
```

Part B: Cross-Validation

Task 4: K-Fold Cross Validation

Perform 5-fold cross-validation.

Report: - Mean accuracy - Standard deviation

```
def evaluate_cv(model, X, y):  
    scores = cross_val_score(model, X, y, cv=5)  
    print(f"Mean Accuracy: {scores.mean():.4f}")  
    print(f"Standard Deviation: {scores.std():.4f}")  
    return scores.mean(), scores.std()
```

Task 5: Comparison

Compare: - Train/Test split - Cross-validation

Discuss differences in performance.

Train/Test Split: Unreliable because it only tests the model on one random 20% slice of data, leading to high variance.

Cross-Validation: Much more reliable because it tests the model on every part of the data.

My Result: My **0.0000 Standard Deviation** proves the model's perfect performance is consistent across all data, likely due to a leakage in the dataset.

Part C: Target Encoding

Task 6: Naive Target Encoding

Encode the categorical feature using mean target value.
Observe model performance.

```
def target_encode(df, column, target):  
    encoding = df.groupby(column)[target].mean()  
    return df[column].map(encoding)
```

Task 7: K-Fold Target Encoding

Implement K-Fold target encoding to avoid leakage.

Compare with naive encoding.

Naive Encoding: Uses the average target value of the entire dataset. This leaks the answer from the test set into the training set.

K-Fold Encoding: Uses the average target value from only the training folds. This is the secure way because it keeps the validation data hidden from the model.

```
def kfold_target_encode(df, column, target, n_splits=5):
    kf = KFold(n_splits=n_splits, shuffle=True, random_state=42)
    encoded_column = pd.Series(index=df.index, dtype=float)

    for train_idx, val_idx in kf.split(df):
        train_fold_mean =
df.iloc[train_idx].groupby(column)[target_col].mean()
        encoded_column.iloc[val_idx] =
df.iloc[val_idx][column].map(train_fold_mean)

    return encoded_column.fillna(df[target_col].mean())
```

Part D: Data Leakage

Task 8: Identify Leakage

Identify features or steps that introduce leakage.

The primary source of leakage is the column named `leak_feature` found in the dataset. My analysis shows this feature has a 99.97% correlation with the target variable. Performing Target Encoding on the entire dataset (as seen in Task 6) before splitting the data is a critical error. This allows the future average values of the test set to leak into the training features.

Explain why they are problematic.

Artificial Performance: It gives the model a cheat sheet, leading to the 1.0 accuracy (100%) you observed.

Real-World Failure: This feature represents information that would not be available at the time of a real prediction (e.g., using a post-diagnosis result to predict the diagnosis itself).

Hidden Vulnerability: In production, the model will look for this cheat and fail completely because the

Task 9: Fix Leakage

Modify the pipeline to remove leakage.

Compare performance before and after.

1. Drop the `leak_feature` column entirely.
2. Use K-Fold Target Encoding so the model only learns from training data, not the validation set.

Comparison:

- Before Fix: 1.0 Accuracy
- After Fix: ~0.80 Accuracy (Real)

Result: The lower score is actually better because the model is now learning real patterns instead of shortcuts.

Part E: Cyclical Features

Task 10: Cyclical Encoding

Transform the hour feature using:

$$\sin \frac{2\pi x}{24}, \quad \cos \frac{2\pi x}{24}$$

```
def encode_cyclical(feature, max_val):  
    sin_feat = np.sin(2 * np.pi * feature / max_val)  
    cos_feat = np.cos(2 * np.pi * feature / max_val)  
    return sin_feat, cos_feat
```

Task 11: Comparison

Compare model performance: - Before encoding - After encoding

Before encoding, the model treats the hour feature as a linear range (0–23), causing a continuity bug where Hour 23 and Hour 0 are seen as polar opposites despite being adjacent neighbors. This linear view makes the model struggle with patterns occurring around midnight. After applying Sine and Cosine transformations, the time is mapped onto a circular unit circle, numerically connecting the end of one day to the start of the next. This After version typically performs better because the model finally understands the physical reality of temporal cycles, leading to higher accuracy and more robust predictions for time-dependent behaviors.

Discussion Questions

- **Why does cross-validation give more reliable estimates than a single split?**

A single split has high variance; it only evaluates one slice of data. CV utilizes 100% of the data for both training and validation across iterations, averaging results to reduce bias.

- **How does target encoding cause leakage?**

Naive target encoding uses the target values of the entire dataset (including the validation/test set) to calculate the mean. This allows information about the outcome to leak into the training features.

- **Why are cyclical features better represented using sine/cosine?**

Raw integers (0-23) suggest that Hour 0 and Hour 23 are numerically opposite. Sine/Cosine mapping places them as adjacent neighbors on a circle, fixing the continuity bug.

- **Which optimization method performed best and why?**

Optuna usually performs best because it is adaptive. It learns from previous trials to focus on promising regions rather than guessing randomly or checking every grid point.